Architecture technique : plateforme de monitoring et d'analyse de logs Smart Building

1. Introduction

Ce document est le livrable « Architecture technique » de la phase Analyse et conception du mini-projet. Il décrit comment la plateforme collecte, indexe, recherche et visualise les logs d'un bâtiment intelligent équipé de capteurs (température, humidité, CO2, luminosité, énergie, occupation).

2. Architecture globale

2.1 Diagramme de contexte

La plateforme est un système logiciel unique, utilisé par trois profils et alimenté par une source externe de données.

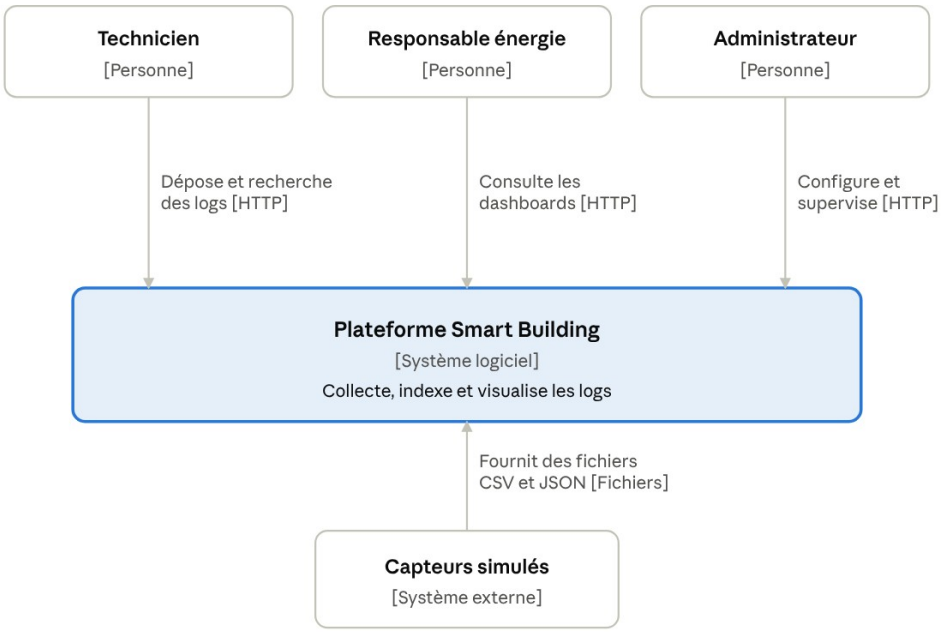


Figure 1 : diagramme de contexte

2.2 Diagramme de conteneurs

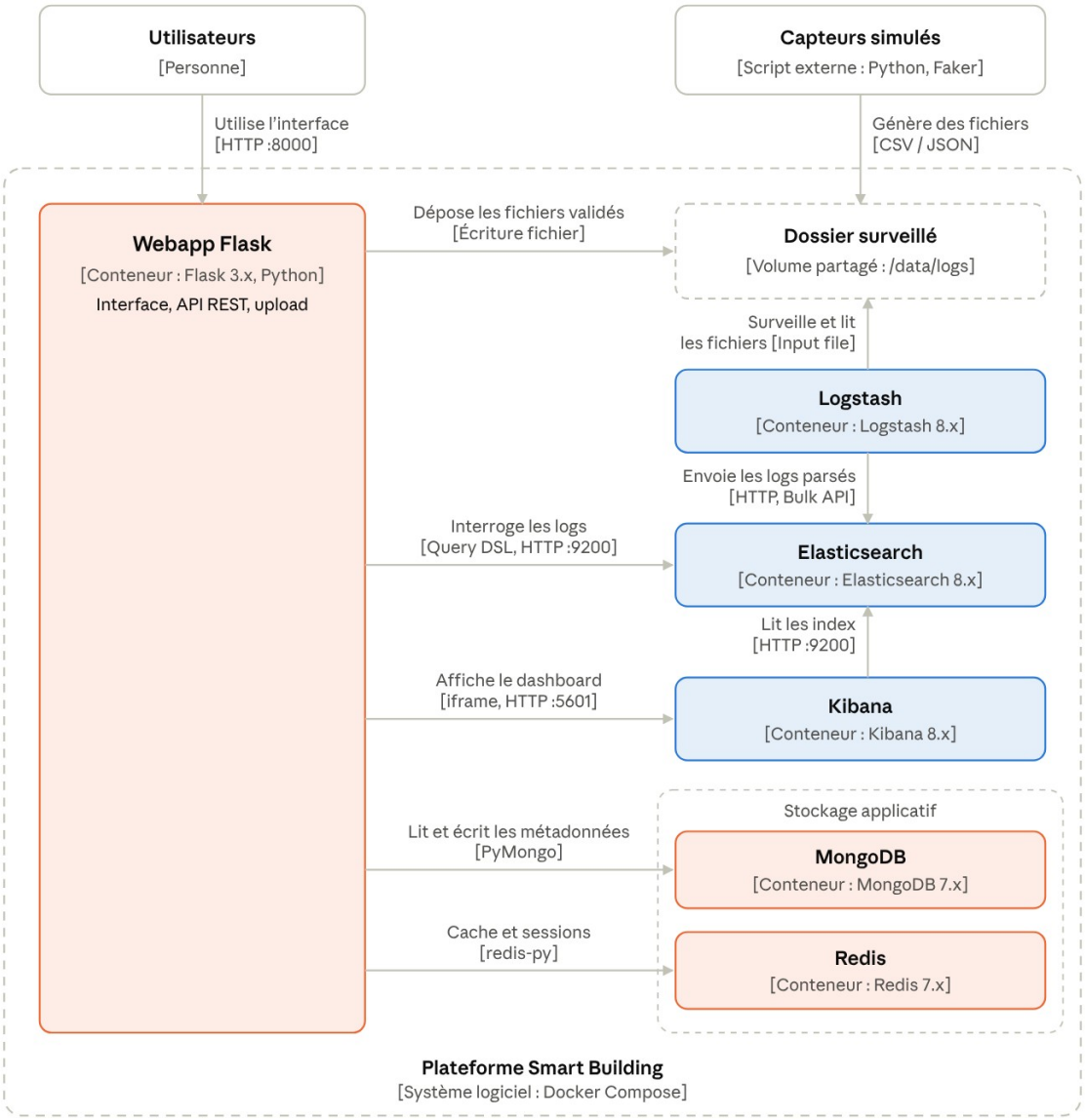


Figure 2 : diagramme de conteneurs

3. Déploiement Docker

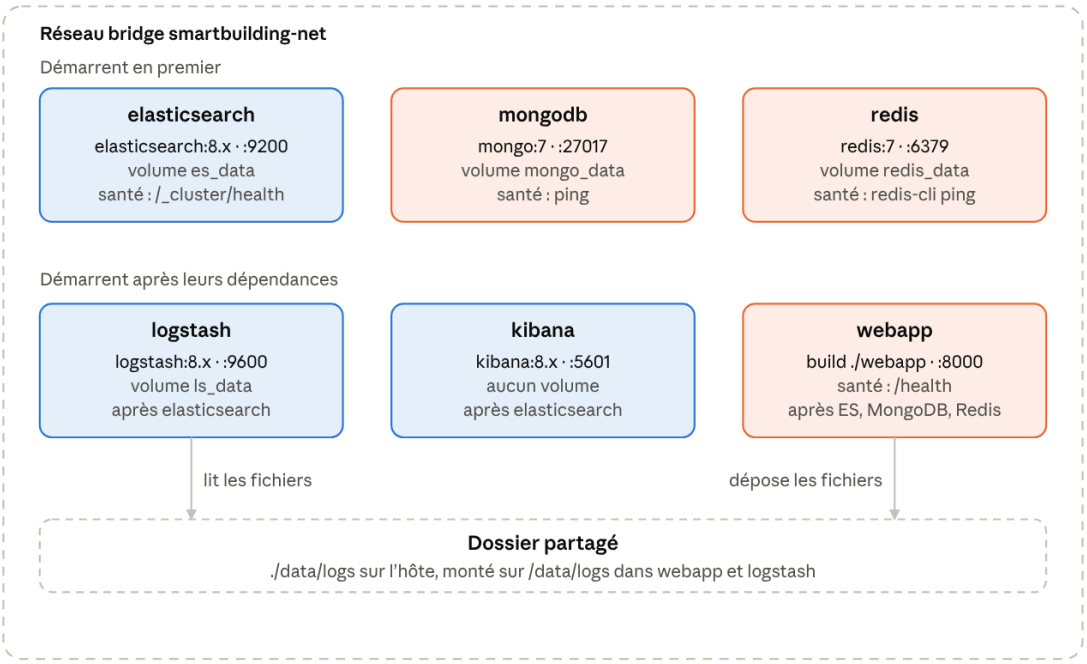


Figure 3 : diagramme de déploiement Docker

4. Diagrammes de séquence

4.1 Upload et ingestion d'un fichier

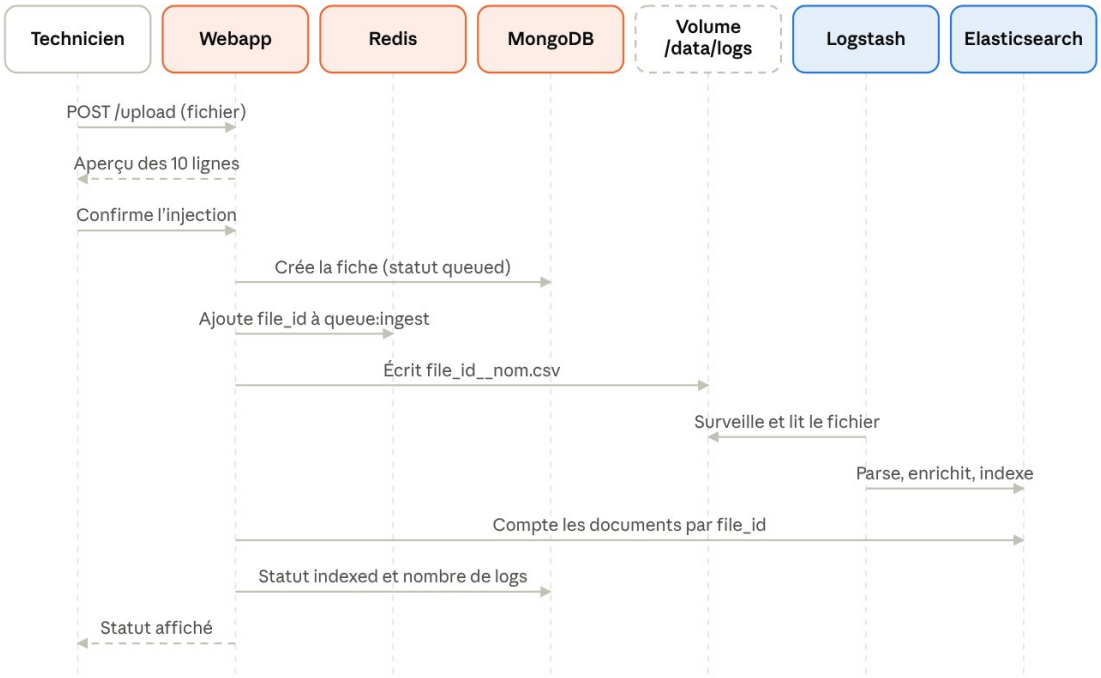


Figure 4 : séquence d'upload jusqu'à l'indexation

4.2 Recherche de logs

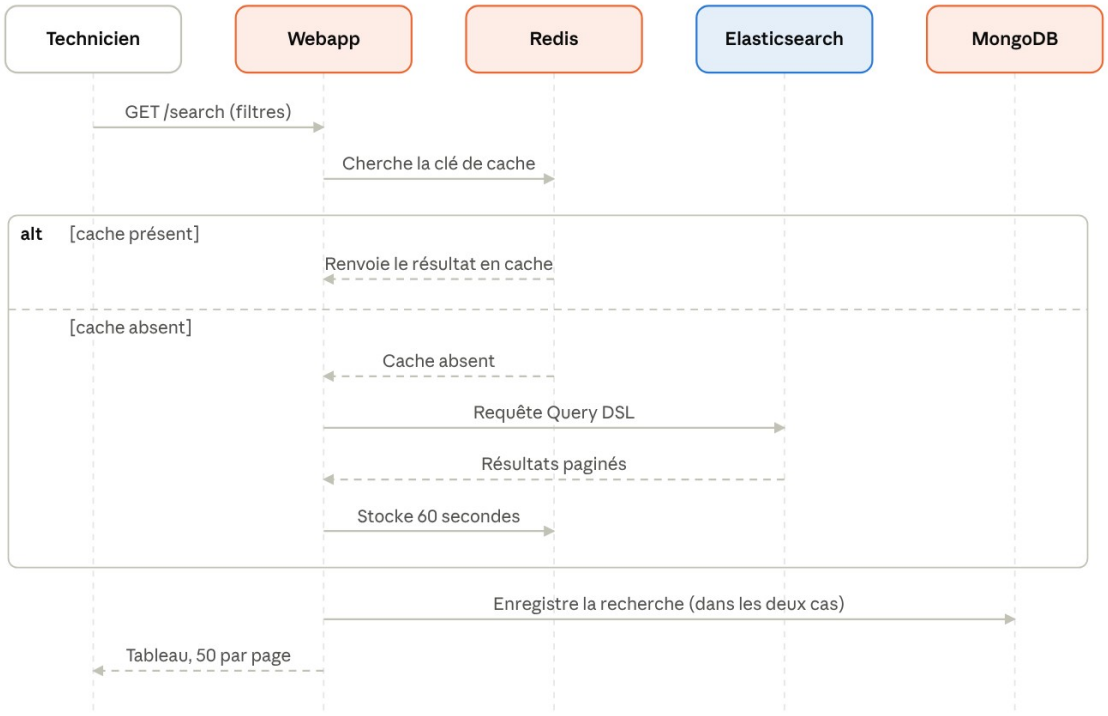


Figure 5 : séquence de recherche avec cache Redis

5. Modèle de données Elasticsearch

Les logs sont indexés par jour, dans des index nommés logs-smartbuilding-AAAA.MM.JJ et couverts par un index template unique. Les documents dont le parsing échoue sont envoyés dans un index séparé, logs-errors-AAAA.MM.JJ, qui conserve le message brut et la cause de l'erreur.

5.1 Champs indexés

Champ	Type	Rôle
`@timestamp`	date	Horodatage de la mesure, au format ISO 8601
`level`	keyword	DEBUG, INFO, WARNING, ERROR ou CRITICAL, calculé par Logstash à l'ingestion
`sensor_type`	keyword	Type de mesure : temperature, humidity, co2, luminosity, energy, occupancy ou alert
`sensor_id`	keyword	Identifiant du capteur
`zone`	keyword	Zone du bâtiment, utilisée pour les filtres et la carte de chaleur

Champ	Type	Rôle
value	float	Valeur mesurée
unit	keyword	Unité de la valeur (°C, %, ppm, lux, kWh)
message	text	Texte libre pour la recherche, avec un sous-champ message.raw en keyword
file_id	keyword	Fichier d'origine, lien avec la fiche MongoDB et comptage des logs indexés
source_file	keyword	Nom du fichier source
tags	keyword	Étiquettes ajoutées par Logstash (format du fichier, résultat du parsing)
ingested_at	date	Date d'indexation par Logstash

Les champs filtrables (level, zone, sensor_type, file_id) sont de type keyword, ce qui permet les filtres exacts et les agrégations des dashboards Kibana. Seul message est analysé en texte pour la recherche libre.

5.2 Index template (mapping JSON)

Mapping proposé, à ajuster pendant le développement. L'option dynamic à false évite l'indexation de champs imprévus.

```
PUT _index_template/logs-smartbuilding
{
  "index_patterns": ["logs-smartbuilding-*"],
  "template": {
    "settings": { "number_of_shards": 1, "number_of_replicas": 0 },
    "mappings": {
      "dynamic": false,
      "properties": {
        "@timestamp":  { "type": "date" },
        "level":       { "type": "keyword" },
        "sensor_type": { "type": "keyword" },
        "sensor_id":   { "type": "keyword" },
        "zone":        { "type": "keyword" },
        "value":       { "type": "float" },
        "unit":        { "type": "keyword" },
        "message":     { "type": "text",
                         "fields": { "raw": { "type": "keyword", "ignore_above": 256 } } },
        "file_id":     { "type": "keyword" },
        "source_file": { "type": "keyword" },
        "tags":        { "type": "keyword" },
        "ingested_at": { "type": "date" }
      }
    }
  }
}
```

5.3 Exemple de document indexé

Exemple fictif : une mesure de CO2 supérieure à 1000 ppm reçoit le niveau WARNING.

```
{
  "@timestamp": "2026-10-14T09:15:30.000Z",
  "level": "WARNING",
  "sensor_type": "co2",
  "sensor_id": "CO2-B2-014",
  "zone": "Salle B2",
  "value": 1180,
  "unit": "ppm",
  "message": "Taux de CO2 élevé dans la salle B2",
  "file_id": "f_20261014_0007",
  "source_file": "capteurs_co2_2026-10-14.csv",
  "tags": [
    "csv",
    "parse_ok"
  ],
  "ingested_at": "2026-10-14T09:16:02.412Z"
}
```

6. Schémas MongoDB

Collection	Rôle	Index
files	Une fiche par fichier envoyé	file_id (unique), uploaded_at, status
searches	Historique des recherches	executed_at, user_id
users	Comptes et rôles (admin, technicien, responsable)	username (unique)

6.1 Exemples de documents

Exemples fictifs pour chaque collection. Le statut d'un fichier prend une des valeurs uploaded, validating, queued, ingesting, indexed ou error. Les mots de passe sont toujours stockés hachés.

Collection files

```
{
  "file_id": "f_20261014_0007",
  "original_name": "capteurs_co2_2026-10-14.csv",
  "stored_name": "f_20261014_0007__capteurs_co2_2026-10-14.csv",
  "extension": "csv",
  "size_bytes": 2483912,
  "uploaded_at": "2026-10-14T09:15:12Z",
  "uploaded_by": "technicien1",
  "status": "indexed",
  "logs_count": 18240,
  "error_message": null
}
```

Collection searches

```
{
  "user_id": "technicien1",
  "executed_at": "2026-10-14T10:02:45Z",
  "query": {
    "text": "capteur défaillant",
    "level": "ERROR",
    "zone": "Salle B2",
    "date_from": "2026-10-13",
    "date_to": "2026-10-14"
  },
  "results_count": 37,
  "from_cache": false
}
```

Collection users

```
{
  "username": "technicien1",
  "password_hash": "<hash du mot de passe>",
  "role": "technicien",
  "active": true,
  "created_at": "2026-10-01T08:00:00Z"
}
```

7. Architecture Redis

Clé	Type	Durée de vie	Usage
cache:search:<hash>	String (JSON)	60 s	Résultat d'une recherche Elasticsearch
cache:stats	String (JSON)	30 s	Indicateurs de la page d'accueil
session:<id>	Hash	durée de la session	Session utilisateur
queue:ingest	List	aucune	File de suivi des fichiers en cours d'ingestion
ratelimit:<ip>:<minute>	String (compteur)	60 s	Limitation du nombre de requêtes API

8. Choix techniques

Choix	Justification
Flask 3.x	Framework léger, suffisant pour une API et quelques pages, rapide à conteneuriser
Dépôt de fichiers lu par Logstash	Découple la webapp de l'ingestion et permet d'ajouter d'autres sources sans la modifier
Index quotidiens	Simplifie la rétention et la suppression des anciennes

Choix	Justification
	données
MongoDB	Schéma souple pour des métadonnées et un historique qui évoluent
Redis	Réponses rapides aux recherches répétées, sessions et compteurs avec expiration automatique
Docker Compose avec healthchecks	Démarrage reproductible en une commande et dans le bon ordre
Bootstrap 5, Chart.js, DataTables	Interface responsive, graphiques et tableaux interactifs sans développement lourd

9. Perspectives

Les évolutions suivantes sont prévues si le temps le permet :

- **Alerting** : règles de seuil sur Elasticsearch, worker séparé, notifications par webhook, délai de répit stocké dans Redis.
- **Temps réel** : WebSocket pour une page de logs en direct et des compteurs en continu.
- **Observabilité** : métriques Prometheus et tableaux Grafana, en complément de l'endpoint /health déjà prévu.